FACULTY OF ENGINEERING AND BASIC SCIENCES
ACADEMIC PROGRAM: DATA ENGINEERING AND ARTIFICIAL INTELLIGENCE

COURSE: ETL (G01)
ETL Project - Second Delivery: Pipeline Automation, API Integration and Data Validation

1. Objective

Evolve the analytical solution created in the First Delivery into a reliable and operational data engineering system. Each team must incorporate a programmatically extracted API source, formulate integrated analytical requirements, automate data-quality validation, refine the dimensional solution, orchestrate the complete workflow with Apache Airflow 3.1.8, and demonstrate controlled behavior during success, failure, and rerun scenarios.

The Second Delivery evaluates the engineering coherence of the complete system. The assessment considers whether requirements, source acquisition, profiling, quality controls, transformation, integration, dimensional modeling, orchestration, observability, and analytics work together and whether the team can justify the decisions that connect them.

This is an independent integrative assessment. The document defines what the system must achieve, justify, and demonstrate. Each team remains responsible for deciding how to implement source-specific extraction, validation, transformation, integration, dimensional refinement, and task organization.

Engineering principle: *A Data Engineer does not build scripts. A Data Engineer designs systems.*

2. Continuity with the First Delivery

The First Delivery is the mandatory baseline for this phase. Teams must preserve the analytical problem, SDG context, principal stakeholders, primary data source, dimensional baseline, and analytical product when these elements remain valid. Changes are allowed only when they correct a documented limitation or are required by the new integrated requirements.
R1–R5 must not be removed arbitrarily. For each requirement, state whether it is maintained, refined, or replaced and provide a concise analytical or design justification for any change.

Requirement	Status	Change Introduced	Justification	Second Delivery Analytical Output
R1	Maintain / Refine / Replace			

Requirement	Status	Change Introduced	Justification	Second Delivery Analytical Output
R2	Maintain / Refine / Replace			
R3	Maintain / Refine / Replace			
R4	Maintain / Refine / Replace			
R5	Maintain / Refine / Replace			

Required evidence: Include a concise baseline summary, a link to the First Delivery repository, the prior dimensional model, and the change decisions recorded in the table above. The First Delivery repository must remain intact.

Replacement is allowed only when the original requirement contains a documented analytical limitation, becomes incompatible with the refined problem, or is superseded by a demonstrably stronger requirement.

3. Second Delivery Scope

The solution must add and connect the following capabilities:

1. A second data source extracted programmatically from an API.
2. At least two new integrated analytical requirements, R6 and R7.
3. Evidence-based profiling of both sources, with emphasis on the new API and on integration risks.
4. Explicit data-quality risks and rules with metrics, thresholds, severity, and requirement traceability.
5. Automated validation with Great Expectations for both raw-source branches and the prepared data.
6. Justified transformation, harmonization, and integration of the two sources.
7. Review and refinement of the dimensional model before final transformation and loading design.
8. A PostgreSQL or MySQL Data Warehouse populated only with accepted prepared data.
9. Complete workflow orchestration with Apache Airflow 3.1.8.
10. Explicit dependency, failure, retry, logging, monitoring, and safe-rerun behavior.
11. A dashboard that queries the Data Warehouse and demonstrates the analytical value of R6 and R7.
12. A new, reproducible, self-contained GitHub repository for the Second Delivery.

4. New API Source

Select an API that is relevant to the Colombian problem and adds information that the primary source does not provide. The API must contribute directly to R6 and R7 and must be part of the operational pipeline.
The API source must be extracted programmatically. Manually downloading a CSV or JSON file and treating it as an API extraction is not accepted. A raw response may be persisted after extraction for reproducibility, inspection, or rerun support, but the operational workflow must contain the API request.

4.1 API Suitability and Technical Documentation

Item	Required Documentation
Provider and purpose	Identify the organization, service, and information supplied by the API.
Analytical value	Explain what the API adds to the problem and why R6 and R7 cannot be answered from the primary source alone.
Endpoint and parameters	Document the endpoint or endpoints, relevant parameters, filters, and the intended extraction scope.
Response	Document the response format, records or objects used, and attributes retained for analysis.
Authentication	Describe the authentication method without exposing credentials.
Operational behavior	Document pagination, rate limits, timeouts, expected errors, and timestamp of extraction when applicable.
Reproducibility	Explain how another authorized evaluator can execute or verify the extraction.

Security requirement: Never commit real API keys, tokens, passwords, or connection strings. Use environment variables or a local .env file and provide an .env.example file with variable names and non-secret examples.

5. New Integrated Analytical Requirements R6 and R7

Define two new analytical requirements identified as R6 and R7. Each requirement must need information from both the primary source and the API. Displaying two separate results, one from each source, does not constitute integration.
R6 and R7 must represent different analytical questions rather than minor variations of the same aggregation. Each requirement must identify the required attributes, their source, the integration strategy, the expected KPI or measure, and the required level of detail.

ID	Analytical Requirement	Primary-Source Data Required	API Data Required	Integration Strategy or Key	Expected KPI or Measure	Required Level of Detail
R6						

ID	Analytical Requirement	Primary-Source Data Required	API Data Required	Integration Strategy or Key	Expected KPI or Measure	Required Level of Detail
R7						

Additional requirements such as R8 or R9 are allowed only when the problem justifies them. Additional requirement count does not receive credit by itself; the assessment prioritizes analytical value, integration, feasibility, and traceability.

Required evidence: Provide the completed R6–R7 matrix, a short justification of the API's analytical value, and a mapping from each required attribute to its source and downstream use.

6. Requirements and End to End Traceability

Strengthen the traceability chain established in the First Delivery. The team must be able to show how each selected requirement influences data acquisition, quality controls, transformation, modeling, and analytical outputs.

Required reasoning chain: *Analytical Requirement → Required Data → Source → Profiling Evidence → Quality Risk → Quality Dimension → Quality Rule → Metric and Threshold → Severity → GX Expectation → Validation Result → Pipeline Action → Transformation and Integration → DW Element → KPI or Visualization*

The final traceability matrix should use representative rows that demonstrate the complete chain. It does not need dozens of repetitive rows.

Requirement	Required Data	Source	Quality Risk	DQ Rule	GX Expectation	Transformation or Integration	DW Element	KPI or Visualization

7. Data Profiling

Profile both data sources. The primary source may reuse valid evidence from the First Delivery, but the team must extend the analysis when R6, R7, or the integration strategy introduces new attributes, quality concerns, temporal constraints, or candidate keys. The API is a new source and requires its own profiling evidence.

Profiling must focus on evidence relevant to the analytical and integration design. Evaluate the following when applicable:

- schema, nested structure, attributes, record counts, and data types;
- completeness, uniqueness, duplication patterns, and candidate keys;
- categorical domains, numerical ranges, unexpected values, and format inconsistencies;

- temporal coverage, date parseability, extraction timestamp, and time granularity;
- cross-source compatibility, integration-key quality, temporal compatibility, and matching coverage;
- attributes required by R1–R7 and evidence that can affect the dimensional model or analytical results.

Outlier analysis is required only when it is relevant to the problem, the variable, and the analytical requirement. A generic outlier procedure without a justified use is not evidence of useful profiling.

Dataset or Attribute	Profiling Evidence	Potential Quality Risk	Related Requirement

Important distinction: *Profiling observes and produces evidence. It must not silently clean or modify the data.*

Required evidence: Submit reproducible profiling code or notebooks, concise result tables or figures, and written interpretations linked to quality risks and requirements. Screenshots alone are not sufficient.

8. Quality Risks and Data Rules

Convert relevant profiling findings, source contracts, integration conditions, and prepared-data invariants into explicit quality risks and rules. Define at least six meaningful rules distributed across the complete solution. Do not force the same number of rules for each source or layer.

Rule ID	Source or Layer	Attribute (s)	Quality Dimension	Quality Rule	Metric or Threshold	Severity	Related Requirement

8.1 Severity and Pipeline Policy

Severity	Meaning	Required Policy Interpretation
Critical	The violation makes normal continuation unsafe.	The affected branch must normally stop, and dependent downstream work must not proceed normally.
Warning	The issue must be recorded and made visible but may not invalidate the batch.	Continuation is allowed only under a documented policy and with preserved evidence.
Informational	The result supports monitoring or trend analysis.	It does not block processing by itself, but the result must remain observable.

Threshold requirement: Explain the analytical, contractual, historical, or operational basis for each material threshold. A threshold selected only because the current data pass is not accepted.

9. Validation Architecture

The pipeline must implement three explicit validation gates:

Validation Gate	Data Evaluated	Question Answered	Downstream Dependency
Primary Source Raw Validation	Extracted primary-source batch	Are the incoming primary-source data safe enough to enter transformation?	Primary branch may proceed only according to the documented policy.
API Raw Validation	Extracted API batch or normalized raw representation	Are the incoming API data safe enough to enter transformation?	API branch may proceed only according to the documented policy.
Prepared Data Validation	Transformed and integrated data prepared for dimensional loading	Did transformation and integration produce data suitable for loading into the analytical Data Warehouse?	The load may proceed only according to the documented policy.

Engineering principle: *Reliable data pipelines do not assume that data is correct. They verify it.*

Great Expectations detects violations and produces validation evidence. The pipeline policy decides whether execution continues. Critical violations must normally block the affected branch; Warning and Informational results follow the documented policy.

10. Automated Validation with Great Expectations

Use Great Expectations to automate the material rules defined for the raw and prepared layers. Every assessed Expectation must be traceable to a DQ Rule ID and to the requirement or invariant it protects.

GX Element	What the Team Must Explain
Data object	Which batch, table, file, or prepared representation is being validated.
Expectation	Which explicit quality rule the Expectation implements and which parameters represent its metric or threshold.
Expectation Suite	Why the grouped checks belong to the same data object and layer.
Validation Definition	How the data definition and Expectation Suite are associated for repeatable execution.
Checkpoint or controlled execution	How validation is invoked and how results become available to the pipeline.
Validation result	How success, failure, unexpected counts or percentages, and relevant examples are interpreted.
Pipeline response	How severity and policy translate the validation result into continue, warn, stop, or another justified action.

Isolated Expectations without an identified risk, rule, threshold, and response are not sufficient.

Required evidence: Preserve the GX configuration or code, suites, validation definitions or controlled execution, and interpretable results for the successful and controlled-failure runs.

11. Raw Extraction and Validation

Each source branch must remain independently observable. Exact task names are not mandatory, but the DAG must expose the following conceptual structure:

Primary Source Branch	API Branch
extract_primary → validate_primary_raw	extract_api → validate_api_raw

- Extraction must preserve enough raw evidence or metadata to identify the evaluated batch.
- Business transformations must not hide source-quality problems before raw validation.
- Each raw validation task must record its result and enforce the documented severity policy.
- Transformation must not begin normally until both required raw branches satisfy their policy.

12. Review and Refinement of the Dimensional Model

Review the dimensional model from the First Delivery before finalizing transformation and integration design. The model must continue to support R1–R5 and must support R6 and R7 after the API source is introduced.

Required design sequence: *Analytical Requirements → Source Understanding and Profiling → Review Existing Dimensional Model → Decide Necessary Refinements → Design Transformation and Integration → Implement and Load the Data Warehouse*

Model Element	Required Review
Business process	Confirm the process or processes represented by the analytical model.
Fact-table grain	State precisely what one row in each Fact Table represents.
Fact table or tables	Explain whether the API enriches an existing fact, supports another fact, or contributes through another justified pattern.
Dimensions	Identify conformed, refined, or new dimensions and explain their analytical role.
Measures	Verify that each measure is compatible with the declared grain and supports a requirement.
Keys and relationships	Document PK, FK, surrogate keys, business keys, and relationship cardinality where applicable.
Requirement coverage	Demonstrate how the model supports R1–R7.

The API does not automatically require a new dimension, a new Fact Table, or a change to the existing grain. Every modification must be justified. A well-supported decision not to modify a specific model element is also valid.

Required evidence: Provide the First Delivery model as baseline, the refined Star Schema, a change log, updated grain statements, and a requirements-to-model validation for R1–R7.

13. Transformation and Integration

Transformation may include cleaning, normalization, harmonization, type conversion, missing-value handling, duplicate handling, derivation, temporal alignment, integration, and construction of analytical attributes when these actions are justified by requirements and known engineering rules.

Engineering principle: *Validation detects. Transformation applies known engineering rules.*
Mandatory condition: *Do not modify data only to make a validation pass.*

For each significant transformation, document the rule or rationale, affected fields, before-and-after behavior, exception handling, and analytical or dimensional impact.

Transformation ID	Rule or Rationale	Affected Fields	Before and After Behavior	Exception Handling	Analytical or Dimensional Impact

13.1 Integration Contract

Treat integration as an engineering contract. Integration may use enrichment, conformed dimensions, new dimensions, fact enrichment, multiple Fact Tables, temporal matching, bridge tables, or another justified dimensional strategy. It does not necessarily mean using pd.merge() on two DataFrames.

Contract Element	Required Decision and Evidence
Integration key or keys	Identify the source attributes, derived keys, or relationship used to connect the sources.
Cardinality	State and verify the expected relationship between matching records.
Preprocessing	Document normalization or standardization required before matching.
Temporal alignment	Explain compatible dates, periods, effective times, or matching windows when relevant.
Unmatched records	Measure and define treatment for records that cannot be integrated.
Duplicate or ambiguous matches	Detect, quantify, and define the response to multiple plausible matches.
Assumptions	State the conditions under which the integration is considered valid.
Limitations	Explain what the integrated data cannot establish reliably.

14. Prepared Data Validation

After transformation and integration, execute a distinct prepared-data validation gate before loading the Data Warehouse.

Required flow: *Prepared Data → validate_prepared → Dimensional Load*

The gate must evaluate the invariants needed for loading and analysis, including the following when relevant:

- expected schema and mandatory fields;
- valid domains, ranges, and derived-attribute rules;

- integration integrity and matching indicators;
- uniqueness at required keys or declared grain;
- referential or key readiness for dimensional loading;
- analytical readiness for R1–R7.

A Critical prepared-data failure must normally prevent the load. The team must preserve the validation result and the Airflow evidence that demonstrates the blocked downstream behavior.

15. Data Warehouse Implementation and Load

Implement the analytical Data Warehouse in PostgreSQL or MySQL. SQLite is not accepted. Any other database technology requires prior instructor approval.

- Create the dimensional structures defined in the refined model.
- Load only data accepted by the prepared-data policy.
- Preserve the declared Fact Table grain and enforce appropriate keys and relationships.
- Load dimensions and facts in an order consistent with referential integrity.
- Provide reconciliation evidence such as accepted input counts, dimension counts, fact counts, unmatched counts, or other relevant controls.

Important distinction: Source persistence, raw copies, and staging structures are not the final analytical Data Warehouse. The final load must write validated data into the dimensional structures used by SQL queries and the dashboard.

16. Apache Airflow Orchestration

Orchestrate the complete workflow with Apache Airflow 3.1.8. Use the TaskFlow API and the public authoring interface shown below.

```
from airflow.sdk import dag, task
```

Exact task names and module organization are team decisions. However, the DAG must make the following capabilities observable:

- primary-source extraction and raw validation;
- API extraction and raw validation;
- transformation and integration after both branches satisfy the policy;
- prepared-data validation;
- dimensional Data Warehouse load;
- states, logs, retries, and failure behavior for the material tasks.

Engineering principle: *First design what the pipeline must guarantee. Then use Airflow to make that process executable, repeatable, and observable.*

16.1 Conceptual DAG Dependencies

Primary Branch	Convergence and Target	API Branch
extract_primary		extract_api
↓		↓
validate_primary_raw	both branches satisfy policy	validate_api_raw
	↓	
	transform_and_clean_and_integrate	
	↓	
	validate_prepared	
	↓	
	load_dimensional_dw	

The diagram is conceptual. Teams may use additional tasks, staging layers, sensors, or grouped operations when they improve the design and remain justified.

17. Task Interfaces

Airflow is the orchestrator, not a bulk-data transport mechanism. Do not use XCom to move complete datasets between tasks. Task interfaces may exchange compact references such as paths, table names, object identifiers, batch identifiers, validation identifiers, or summary metadata.

Interface Concern	Required Decision
Input contract	What reference, parameter, or persisted object does the task require?
Output contract	What persisted artifact or compact metadata does the task produce?
Batch identity	How is the same source batch recognized across extraction, validation, transformation, and load?
Failure signal	How does the task expose a failure that Airflow can record and enforce through dependencies?
Rerun behavior	What happens when the same task or batch executes again?

18. Failure Handling and Retry Policy

Document the failure policy for material tasks and conditions. A reliable system must distinguish failures that may disappear on another attempt from failures caused by deterministic data or contract violations.

Condition	Type or Severity	Pipeline Response	Retry	Justification

Failure Class	Examples	Expected Retry Reasoning
Transient operational failure	Temporary API unavailability, timeout, temporary network problem, temporary database connection problem.	A bounded retry may be justified because the same input and code may succeed on a later attempt.
Deterministic data or contract failure	Invalid schema, missing mandatory attribute, Critical GX failure, incompatible integration keys.	Automatic repetition normally reproduces the failure. Stop, preserve evidence, and require an explicit correction or recovery path.

A blanket policy such as retries=3 everywhere is not accepted as evidence of reliability. Justify retry count, delay, and scope wherever retries are configured.

Reliability principle: *Reliable does not mean that nothing fails. Reliable means that failures are anticipated, controlled, and observable.*

19. Logging and Observability

Use task states, validation results, timestamps, and logs to answer the following questions for each material run:

Operational Question	Expected Evidence
What ran?	DAG Run, Task Instance, source or branch, and relevant operation.
When did it run?	Logical or run date and material start and completion timestamps.
Which batch or source?	Batch identifier, extraction timestamp, path, table, or compact source reference.
How many records were processed?	Relevant extracted, accepted, rejected, matched, or loaded counts.
Did validation pass?	Overall and relevant rule-level GX outcomes.
Where and why did failure occur?	Failed task state and actionable log or validation evidence.

Operational Question	Expected Evidence
What happened next?	Retry, blocked dependency, warning continuation, recovery, or other policy response.

Required evidence: Use the Airflow Web UI to show the DAG structure, DAG Runs, task states, configured retry behavior when applicable, and failed-task logs. Each screenshot must be legible, cropped to the relevant evidence, and accompanied by an interpretation.

20. Mandatory Reliability Tests

20.1 Test A Successful Run

Execute the complete pipeline using a batch that satisfies the Critical policy. Demonstrate both extractions, both raw validation gates, transformation and integration, prepared validation, dimensional load, Data Warehouse data, and analytical results.

Stage	Required Evidence
Source acquisition	Primary-source and API extraction states, source or batch identity, and relevant record counts.
Raw validation	Successful policy result for both branches and interpretable GX evidence.
Transformation and integration	Successful task state plus reconciliation or matching evidence.
Prepared validation	Accepted prepared batch and relevant GX result.
Load and analytics	Loaded dimensional data, query results, and dashboard outputs.

20.2 Test B Controlled Critical Quality Failure

Introduce or activate a controlled condition that violates one justified Critical rule. The team chooses the rule and failure condition; the brief does not prescribe a specific value to modify.

Expected behavior: Extraction may succeed, the affected validation task fails, dependent downstream processing does not proceed normally, the load does not proceed normally, and the failure remains visible and diagnosable.

- Airflow Graph or Grid evidence;
- failed Task Instance state;
- Great Expectations validation result;
- failed-task log;
- explanation of the violated rule, its Critical severity, and the affected requirement;
- explanation of downstream task behavior.

20.3 Test C Safe Rerun and Repeatability

Execute the same input batch again. The rerun must not silently create unintended duplicates or corrupt the Data Warehouse.

Document the selected strategy, such as replace, truncate-and-load, controlled upsert, business-key enforcement, batch identifiers, or another justified approach. Compare relevant target counts, keys, and measures before and after the rerun and explain how the design behaves after a partial or failed run.

Mandatory condition: *A rerun must not silently corrupt the analytical target.*

21. Business Intelligence and Analytical Product

Maintain the First Delivery analytical solution when it remains relevant and extend it to demonstrate the value created by the API integration. The dashboard must query the PostgreSQL or MySQL Data Warehouse. Direct connections to CSV files, raw API responses, or intermediate files are not accepted as the evaluated solution.

- R1–R5 must remain represented when they continue to be valid.
- R6 and R7 must be reflected in the final analytical product.
- At least one KPI and one visualization must clearly demonstrate value created by integrating the API source.
- Every KPI or visualization must identify the requirement and Data Warehouse elements that support it.

Requirement	DW Element or Query	KPI or Visualization	Integration Value Demonstrated
R6			
R7			

22. System Architecture

Provide a high-level architecture diagram that represents components, persistence layers, validation gates, orchestration, and data flow. The architecture must include, at minimum:

- primary data source and programmatic API source;
- raw extraction or persistence used by each branch;
- primary and API raw-validation gates;
- transformation and integration;
- prepared-data validation;
- PostgreSQL or MySQL dimensional Data Warehouse;
- SQL queries, KPIs, and dashboard;
- Apache Airflow orchestration, dependencies, retries, logs, and monitoring.

The diagram should communicate system responsibilities and data movement. It must not be a screenshot of the source code or a substitute for the Airflow DAG evidence.

23. New GitHub Repository

Create a new repository for the Second Delivery. Do not overwrite or reuse the First Delivery repository as the primary repository for this phase. The new repository preserves the first phase as evidence, supports comparison, and provides an independent reproducible solution.

A naming convention such as etl-project-delivery-2-team-X is recommended but not mandatory. The repository must clearly identify the Second Delivery and must reference the First Delivery repository. The new repository must be self-contained for assessment. An evaluator must not need to open files manually from the old repository to configure or execute the Second Delivery.

23.1 Suggested Repository Organization

The following organization is suggested rather than imposed. Equivalent structures are valid when they preserve separation of concerns, reproducibility, and clarity.

```
etl-project-delivery-2-team-X/
├── dags/
├── src/
├── gx/
├── notebooks/
├── sql/
├── docs/
│   └── evidence/
├── data/ or documented external data location
├── requirements.txt
├── Dockerfile
├── docker-compose.yaml
├── .env.example
├── .gitignore
└── README.md
```

Organize orchestration logic separately from reusable extraction, validation, transformation, integration, and loading logic when practical. Do not commit secrets, virtual environments, generated caches, or unnecessary large files.

24. README

The README must function as the technical documentation for the complete Second Delivery solution and must include:

1. Problem and analytical objective.
2. First Delivery baseline and repository link.

3. Status and changes for R1–R5.
4. New integrated requirements R6 and R7.
5. Primary source and API source.
6. Analytical value contributed by the API.
7. System architecture.
8. Data profiling findings.
9. Quality risks and quality rules.
10. Great Expectations design and rule traceability.
11. Raw validation design for both sources.
12. Transformation strategy and integration contract.
13. Refined dimensional model and change justification.
14. Prepared-data validation.
15. Data Warehouse implementation and load.
16. Airflow DAG design and task interfaces.
17. Failure and retry policies.
18. Successful-run evidence.
19. Controlled-failure evidence.
20. Repeatability and safe-rerun strategy.
21. Dashboard and analytical outputs.
22. Requirement-to-KPI and end-to-end traceability.
23. Setup and execution instructions.
24. Assumptions and limitations.
25. Team members and responsibilities.

25. Minimum Evidence Set

Organize the evidence inside the repository and reference it from the README. At minimum, include:

- ☐ R1–R5 status and refined analytical requirements.
- ☐ Completed R6–R7 integrated-requirements matrix.
- ☐ Architecture diagram.
- ☐ Profiling evidence for both sources.
- ☐ Quality risks and completed quality-rule table.
- ☐ Programmatic API extraction evidence.
- ☐ Great Expectations raw-validation evidence for both branches.
- ☐ Transformation and integration evidence, including the integration contract.
- ☐ Refined dimensional model and Data Warehouse scripts.
- ☐ Prepared-data validation evidence.
- ☐ Airflow DAG and successful-run evidence.
- ☐ Controlled-failure evidence and failed-task logs.
- ☐ Retry-policy evidence or justified no-retry decisions.
- ☐ Safe-rerun evidence and before-and-after comparison.
- ☐ Dashboard evidence and requirement-to-KPI traceability.
- ☐ Final end-to-end traceability matrix.

Evidence standard: Screenshots must be legible, cropped to the relevant information, and accompanied by an interpretation. A screenshot without a stated claim and explanation is not sufficient evidence.

26. Deliverable

Submit only the URL of the new Second Delivery GitHub repository through the LMS. The repository must contain or reference all required deliverables and must be accessible to the instructor according to the course submission procedure.

Deliverable	Required Contents
1. New GitHub Repository	Independent, self-contained Second Delivery repository with a reference to the First Delivery repository.
2. Source Code	Reusable extraction, validation, transformation, integration, and loading logic.
3. Airflow DAG	Complete orchestration, dependencies, selective retries, failure behavior, and observability.
4. API Extraction	Programmatic extraction, configuration, documentation, and raw evidence or metadata.
5. Great Expectations	Configuration or code, suites, validation definitions or controlled execution, and results.
6. Profiling and Quality Design	Profiling code and evidence, quality risks, quality rules, metrics, thresholds, severity, and traceability.
7. Dimensional Data Warehouse	Refined model, schema scripts, load logic, keys, constraints, and reconciliation evidence.
8. Reliability Evidence	Successful run, controlled Critical failure, logs, retry decisions, and safe rerun.
9. Analytical Product	Dashboard connected to the Data Warehouse with R6–R7 outputs and API-integration value.
10. Technical Documentation	README, architecture, integration contract, matrices, assumptions, limitations, and evidence folder.
11. Environment Files	requirements.txt, Dockerfile, docker-compose.yaml when applicable, .env.example, and .gitignore.
12. Presentation and Demo	Material and prepared environment required to explain and demonstrate the implemented solution.

27. PRESENTATION AND TECHNICAL DEFENSE

The team will present and defend the implemented solution. Every member must understand the analytical problem, First Delivery baseline, R1–R7, source acquisition, API value, profiling, quality

rules, Great Expectations design, transformation, integration contract, dimensional model, Airflow DAG, failure policy, retries, safe rerun, Data Warehouse, dashboard, and limitations.

The presentation must include:

- architecture and major engineering decisions;
- R6 and R7 and the analytical value created by the API;
- major profiling findings, quality risks, rules, thresholds, and severity;
- successful end-to-end execution;
- controlled Critical failure and downstream behavior;
- safe-rerun evidence;
- refined dimensional model and dashboard results.

During the defense, any team member may be asked to explain why a rule exists, how a threshold was justified, why a failure blocks or does not block execution, why a retry is appropriate, how integration limitations affect analysis, how reruns avoid duplicate effects, or how the dimensional model supports R1–R7.

Demonstration condition: The instructor may request live execution or inspection of selected components. The team must preserve prior evidence in case a remote API or external dependency is temporarily unavailable during the presentation.

28. Evaluation

The final grade for this delivery is 5.00 points. The engineering solution and repository account for 4.00 points (80%), and the presentation and technical defense account for 1.00 point (20%).

28.1 Engineering Solution and Repository (4.00 points)

Criterion	What Is Evaluated	Points
Analytical Requirements, API Value and Architecture	Continuity of R1–R5; quality of R6–R7; API relevance; source and system architecture; end-to-end traceability.	0.45
Profiling, Quality Risks and Quality Rules	Relevant profiling evidence; integration risks; explicit dimensions, metrics, thresholds, severity, and justification.	0.55
Extraction, Transformation and API Integration	Reproducible source acquisition; programmatic API extraction; justified transformation; integration contract; reconciliation and limitations.	0.70
Automated Validation with Great Expectations	Meaningful and traceable Expectations; both raw gates; prepared gate; interpretable results; documented pipeline response.	0.75
Airflow Orchestration and Reliability	Complete DAG; dependencies; task interfaces; selective retries; logs; successful run; controlled failure; safe rerun and observable behavior.	0.85

Criterion	What Is Evaluated	Points
Dimensional Data Warehouse and Analytics	Justified model refinement; grain, facts, dimensions, keys and measures; validated load; R1–R7 support; DW-backed dashboard and API-integration value.	0.45
Repository, Reproducibility and Documentation	New self-contained repository; environment files; security hygiene; README; evidence organization; clarity and reproducibility.	0.25
TOTAL		4.00

28.2 Presentation and Technical Defense (1.00 point)

Criterion	Points
Architecture and engineering decisions	0.35
Successful and controlled-failure demonstration	0.30
Technical understanding and defense	0.25
Communication and professionalism	0.10
TOTAL	1.00

Final total: *Engineering Solution and Repository 4.00 + Presentation and Technical Defense 1.00 = 5.00 points.*

Evaluation principle: Functional correctness asks whether the pipeline executes. Engineering reliability asks whether the system behaves correctly, visibly, and repeatably when something goes wrong. A solution that demonstrates only the successful path cannot receive a high reliability score.

29. Final Submission Review

Before submitting, verify that the repository demonstrates every condition below:

☐ R1–R5 continue from the First Delivery or have documented justified refinements.
☐ R6 and R7 each require data from both the primary source and the API.
☐ The API contributes real analytical value and is extracted programmatically.
☐ Both source branches have raw validation and prepared data have a separate validation gate.
☐ Profiling is separated from transformation, and validation is separated from cleaning.
☐ DQ rules include metrics or thresholds, severity, justification, and requirement traceability.
☐ The dimensional model is reviewed before final transformation and integration design.
☐ Airflow 3.1.8 represents the complete workflow and enforces the required dependencies.
☐ Critical failures block unsafe downstream processing and remain observable.
☐ Retries are selective and justified.

- ☐ The successful, controlled-failure, and safe-rerun tests are documented.
- ☐ The dashboard queries the Data Warehouse and represents R6 and R7.
- ☐ At least one KPI and one visualization demonstrate the value of API integration.
- ☐ The Second Delivery uses a new self-contained repository and leaves the First Delivery repository intact.
- ☐ The submitted work defines and justifies engineering decisions without relying on undocumented manual steps.